

NATIONAL CYBER EMERGENCY RESPONSE TEAM OF PAKISTAN (PKCERT)

Government of Pakistan

API SECURITY ASSESSMENT REPORT

Document Version	1.0
Document Name	VAmPI – API Security Assessment Report (Week 7)
Document No	PKCERT/INTERN-STT/2026/WK7/001
Classification	CONFIDENTIAL
Issuing Authority	National CERT of Pakistan (PKCERT) — Internship Program
Date of Issuance	4 September 2026

Important Note

This report reflects the security posture of the assessed API only at the time of assessment. Any subsequent modification, upgrade, configuration change, or functional alteration to the API may affect its security posture and, therefore, may render the findings and conclusions of this report invalid.

This assessment was carried out as part of a supervised internship exercise on a deliberately vulnerable training API (VAmPI — Vulnerable API).

Executive Summary

This report covers Week 7 of the internship: API Security — Mapping and Automated Testing. The target used for this exercise was VAmPI (Vulnerable API), a REST API intentionally built with common API security weaknesses, documented through an OpenAPI (Swagger) specification, for training purposes.

The assessment followed the full workflow required by the task: the API's endpoints were first mapped using its OpenAPI specification, imported into both Postman and OWASP ZAP. Postman was then used to manually walk through every endpoint, test the registration and login (authentication) flow, and probe access control between different user accounts. OWASP ZAP was used, working from the same OpenAPI definition and from authenticated traffic proxied through it from Postman, to run an automated scan of the API.

Manual testing with Postman uncovered four Critical-severity issues: an unauthenticated endpoint that resets the entire database, a Broken Object Level Authorization (BOLA) flaw that allows any logged-in user to read another user's private data, a Mass Assignment vulnerability that lets any new user grant themselves administrator privileges simply by adding an extra field during registration, and an Excessive Data Exposure issue where the API returns every user's password in plain text.

1. Summary of Findings

1.1 Basic Information

Target Owner	PKCERT — Internal Training Lab
Target Custodian	Hafiz Muhammad Moawiz (Trainee, Software Security Testing)
Target Name	VAmPI — Vulnerable API
Target Type	REST API, documented via OpenAPI 3.0 (Swagger) specification
Target URL	http://localhost:5000 (hosted in an isolated lab VM via Docker)
Assessment Type	API Security Assessment — Endpoint Mapping, Manual Testing, and Automated Scanning
Assessment Date	4 September 2026
Tools Used	Postman, Swagger/OpenAPI 3.0 specification, OWASP ZAP (API import + Active Scan)

1.2 Consolidated Vulnerability Summary

Severity	Count	% of Total
Critical	4	50.0%
High	0	0%
Medium	1	12.5%
Low / Informational	3	37.5%
TOTAL	8	100%

1.3 Consolidated Risk Overview

The overall risk rating for this API is Critical. Multiple endpoints either require no authentication at all for sensitive or destructive actions, or perform authentication without properly checking authorization — that is, the API correctly identifies who a user is, but does not consistently check what that user is allowed to do or see. This pattern (present across four separate Critical findings) indicates a systemic gap in access control design rather than an isolated coding mistake.

The most severe combination of findings is that a completely new, self-registered user can (a) grant themselves administrator privileges through Mass Assignment, and (b) the API's own documentation shows a DELETE /users/v1/{username} endpoint restricted to “Only Admins”. Although deletion was not actively tested in this assessment to avoid disrupting the shared lab environment, the combination of these two findings means a self-escalated administrator account would very likely be able to delete any other user in the system.

1.4 API Endpoint Map

The table below lists every endpoint identified from the VAmPI OpenAPI specification, mapped using Postman and cross-checked against the Swagger UI documentation, together with whether authentication is required and what was found during testing.

Method	Endpoint	Auth Required?	Purpose
GET	/	No	API home / info
GET	/createdb	No	Resets and repopulates database
GET	/books/v1	No	Lists all books and owners
POST	/books/v1	Yes	Add new book
GET	/books/v1/{book_title}	Yes (weak)	Retrieves book + secret by title
GET	/me	Yes	Retrieves current logged-in user
GET	/users/v1	No	Lists all users, including passwords
GET	/users/v1/_debug	No	Retrieves full details for all users
POST	/users/v1/login	No (by design)	Login, returns JWT auth token
POST	/users/v1/register	No (by design)	Register new user (accepts unexpected “admin” field)
GET	/users/v1/{username}	Yes	Retrieves user by username
PUT	/users/v1/{username}/email	Yes	Update user's email
PUT	/users/v1/{username}/password	Yes	Update user's password
DELETE	/users/v1/{username}	Yes (Admin only)	Deletes user by username

2. Vulnerability Assessment

Testing followed a three-stage approach: (1) the API's OpenAPI/Swagger specification was imported into Postman to generate a complete, organized collection of every endpoint; (2) each endpoint was then exercised manually in Postman, first without authentication and then using a valid login token, specifically probing the authentication flow and testing for access control weaknesses between different user accounts; (3) the same OpenAPI specification was imported into OWASP ZAP, and an automated Active Scan was run, including against authenticated traffic captured by proxying Postman's requests through ZAP.

2.0 Findings Summary

#	Finding	Severity	Status
F1	Unauthenticated Database Reset (GET /createdb)	Critical	Confirmed
F2	Broken Object Level Authorization (BOLA) — GET /books/v1/{book_title}	Critical	Confirmed
F3	Mass Assignment — Self-Privilege Escalation via Registration	Critical	Confirmed
F4	Excessive Data Exposure — Plaintext Passwords Returned for All Users	Critical	Confirmed
F5	Unauthenticated Disclosure of Book Ownership Data (GET /books/v1)	Medium	Confirmed
F6	Missing Security Headers (Server Version Disclosure, X-Content-Type-Options)	Low	Confirmed
F7	Extremely Short JWT Token Expiry (~60 seconds)	Informational	Observation
F8	Elevated Impact — Self-Escalated Admin Combined with Admin-Only Delete Endpoint	Informational	Impact Note (Not Actively Exploited)

Detailed steps, results, and evidence for each finding are given below.

2.1 Unauthenticated Database Reset (GET /createdb)

Severity: Critical **Status:** Confirmed

What we did:

The OpenAPI specification listed a /createdb endpoint under a “db-init” category. This was called directly in Postman with no Authorization header set, to check whether it required any credentials.

- Sent GET {{baseUrl}}/createdb with no authentication.

What we found:

The request succeeded with 200 OK and the response {"message": "Database populated."}. No login, token, or API key of any kind was required.

Impact:

Any unauthenticated party who can reach this API can reset and repopulate its entire database on demand. In a production system, an equivalent administrative/destructive endpoint left unauthenticated could allow denial of service or destruction of live data by any anonymous user.

Evidence: Figure 3

2.2 Broken Object Level Authorization (BOLA) — GET /books/v1/{book_title}

Severity: Critical **Status:** Confirmed

What we did:

A normal user account (name1) was registered and logged in to obtain a valid JWT token. The unauthenticated /books/v1 endpoint had already revealed that a book titled bookTitle83 belongs to a different user, admin. The test was to see whether name1's own token could be used to retrieve admin's book.

- Logged in as name1 / pass1 via POST /users/v1/login and captured the returned auth_token.
- Sent GET /books/v1/bookTitle83 with no Authorization header first, to confirm the endpoint is protected — received 401 Unauthorized ("Invalid token. Please log in again.").
- Repeated the same request, this time with Authorization: Bearer <name1's token> in the Headers.

What we found:

The request succeeded with 200 OK, returning: {"book_title": "bookTitle83", "owner": "admin", "secret": "secret for bookTitle83"}. The API correctly required a valid token, but never checked whether the token's owner (name1) actually owned the requested resource (admin's book).

Impact:

Any authenticated user, regardless of their own privilege level, can read any other user's private data simply by knowing or guessing the resource identifier (here, the book title). This is API1:2023 on the OWASP API Security Top 10 (Broken Object Level Authorization) and is one of the most common and severe classes of API vulnerability.

Evidence: Figures 7 and 8

2.3 Mass Assignment — Self-Privilege Escalation via Registration

Severity: Critical **Status:** Confirmed

What we did:

The /users/v1/register endpoint's documented schema only expects username, password, and email. To test for Mass Assignment, an additional, undocumented field (admin) was added to the registration request body to see if the API would blindly accept and store it.

- Sent POST /users/v1/register with body: {"username": "hackeruser", "password": "Test@123", "email": "hacker@test.com", "admin": true}.
- Received 200 OK: {"message": "Successfully registered. Login to receive an auth token."}.
- Logged in as hackeruser / Test@123 via POST /users/v1/login and received a valid auth_token.

What we found:

A subsequent listing of all users (see Finding F4) confirmed that the newly created hackeruser account was stored in the database with "admin": true — identical in privilege to the pre-existing admin account. The extra field was accepted with no validation or filtering.

Impact:

Any anonymous visitor can register their own account and grant it administrator privileges in the same request, completely bypassing any intended privilege model. Combined with Finding F8, this could allow a brand-new, self-created account to perform admin-only actions such as deleting other users.

Evidence: Login response for hackeruser, and the /users/v1 listing in Finding F4

2.4 Excessive Data Exposure — Plaintext Passwords Returned for All Users

Severity: Critical Status: Confirmed

What we did:

While investigating the result of the Mass Assignment test (Finding F3), the /users/v1 endpoint (which lists all users) was queried to confirm hackeruser's stored privilege level.

- Sent GET /users/v1.

What we found:

The response listed every user in the system, including full plaintext passwords for each one, for example:

```
{
  "users": [
    {"admin": false, "email": "mail1@mail.com", "password": "pass1", "username": "name1"},
    {"admin": false, "email": "mail2@mail.com", "password": "pass2", "username": "name2"},
    {"admin": true, "email": "admin@mail.com", "password": "pass1", "username": "admin"},
    {"admin": true, "email": "hacker@test.com", "password": "Test@123", "username":
    "hackeruser"}
  ]
}
```

Impact:

Two distinct issues are present here: first, the API returns a sensitive field (password) that should never be included in any user-listing response; second, the passwords are stored and returned in plain text rather than as a salted hash, meaning the API's own backend is not applying any password protection at all. Any party able to call this endpoint obtains every user's real, working password.

Evidence: Figure 5 shows the equivalent unauthenticated listing behaviour on /books/v1; the /users/v1 response for this finding is reproduced below.

2.5 Unauthenticated Disclosure of Book Ownership Data (GET /books/v1)

Severity: Medium Status: Confirmed

What we did:

The /books/v1 endpoint (list all books) was called with no authentication to check whether it required a login, given that the more specific /books/v1/{book_title} endpoint did enforce a token.

- Sent GET /books/v1 with no Authorization header.

What we found:

The request succeeded with 200 OK, listing every book title together with its owner's username (e.g. bookTitle83 → admin, bookTitle33 → name1, bookTitle18 → name2), with no authentication required at all.

Impact:

While this endpoint does not directly expose the “secret” contents of each book, it does leak which usernames exist in the system and what resources they own, and it directly enabled the BOLA test in

Finding F2 by revealing that bookTitle83 belongs to admin. Inconsistent authentication requirements between similar endpoints (list vs. detail) is itself a design weakness.

Evidence: Figure 6

2.6 Missing Security Headers (Server Version Disclosure, X-Content-Type-Options)

Severity: Low **Status:** Confirmed

What we did:

An OWASP ZAP Active Scan was run against the API's endpoints, using the imported OpenAPI definition and authenticated traffic captured by proxying Postman's requests through ZAP.

- Imported the OpenAPI specification into ZAP and confirmed the endpoint tree matched the one seen in Postman and Swagger UI.
- Configured Postman to route its traffic through ZAP's local proxy so that authenticated (logged-in) requests would also be scanned, not just anonymous ones.
- Ran an Active Scan against the API's endpoints, including /books/v1/bookTitle83 and /me.

What we found:

ZAP's Alerts tab flagged, across multiple endpoints: "Server Leaks Version Information via 'Server' HTTP Response Header Field", and "X-Content-Type-Options Header Missing".

Impact:

These are lower-severity configuration weaknesses on their own, but they make the server's technology stack easier to fingerprint and slightly increase the risk of content-type confusion (MIME-sniffing) attacks against API clients that render responses as HTML.

Evidence: Figures 10, 11 and 12

2.7 Extremely Short JWT Token Expiry (~60 seconds)

Severity: Informational **Status:** Observation

What we did:

While testing the BOLA finding (F2), a login token that had worked moments earlier suddenly returned 401 Unauthorized ("Signature expired. Please log in again.") on a retry.

- Decoded the JWT payload obtained at login and compared its iat (issued-at) and exp (expiry) timestamp claims.

What we found:

The token's exp value was only 60 seconds after its iat value — the token is valid for approximately one minute before it must be renewed via a fresh login.

Impact:

A token lifetime this short is unusual and, while shorter-lived tokens are generally a good security practice, an extremely aggressive expiry can encourage insecure workarounds in real client applications (such as caching credentials to auto-relogin) and made manual testing noticeably harder, since requests had to be sent within seconds of obtaining a token.

Evidence: No image — observed via token payload timestamps during testing

2.8 Elevated Impact — Self-Escalated Admin Combined with Admin-Only Delete Endpoint

Severity: Informational **Status:** Impact Note (Not Actively Exploited)

What we did:

The Swagger UI documentation for the API was reviewed to identify any endpoints restricted to administrators, to assess the real-world impact of the Mass Assignment finding (F3).

- Reviewed the full endpoint list in Swagger UI (<http://localhost:5000/ui/>).

What we found:

The documentation explicitly lists DELETE /users/v1/{username} as “Deletes user by username (Only Admins)”. This endpoint was not actively called with the self-escalated hackeruser account in this assessment, to avoid disrupting user accounts in the shared lab environment, so this finding is recorded as an impact note rather than an actively confirmed exploit.

Impact:

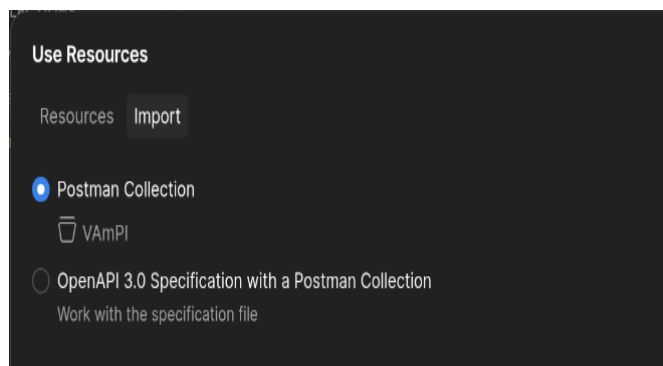
Given that Finding F3 already demonstrated that any anonymous user can register an account with "admin": true accepted at face value, it is highly likely that the same self-escalated account could also successfully call this admin-only delete endpoint against any other user, including the legitimate admin account. This should be verified and prioritised for remediation alongside Finding F3.

Evidence: Figure 14

3. Proof-of-Concept Evidence

All screenshots below were captured live during Week 7 testing and correspond directly to the findings described in Section 2.

Figure 1 — OpenAPI Specification Imported into Postman



VAmPI's OpenAPI 3.0 specification imported into Postman as a ready-to-use collection.

Complete VAmPI endpoint tree generated automatically from the OpenAPI specification, used as the basis for the API map in Section 1.4.

Figure 2 Generated Postman Collection (Full Endpoint Map)

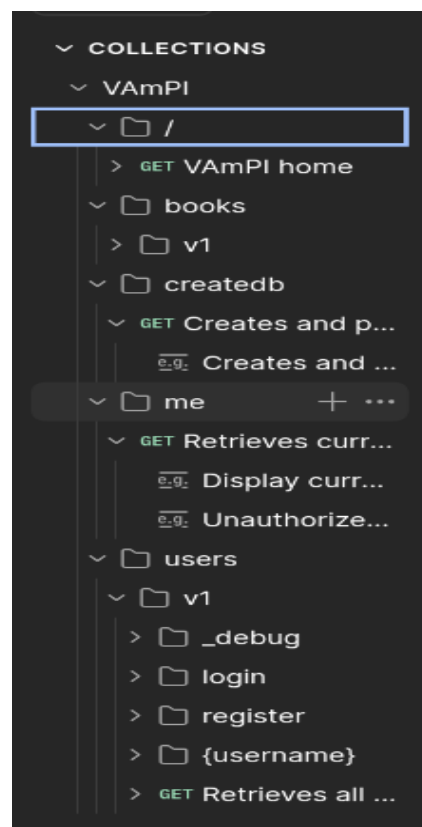
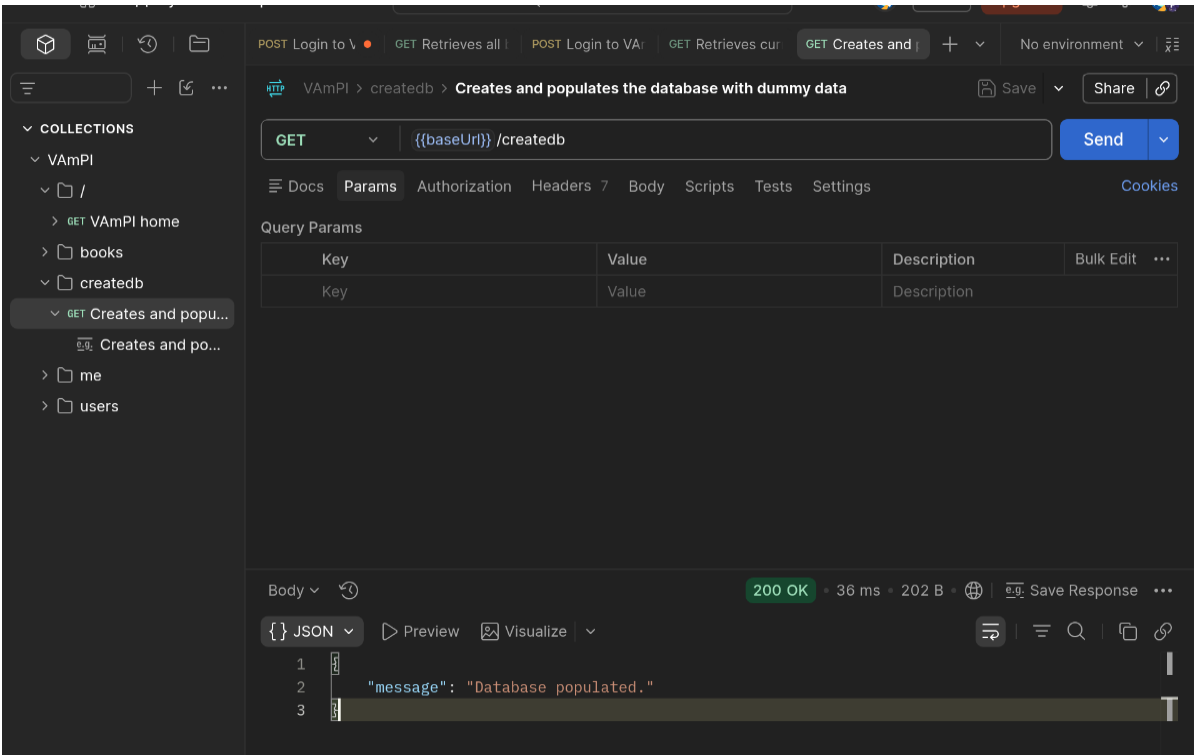
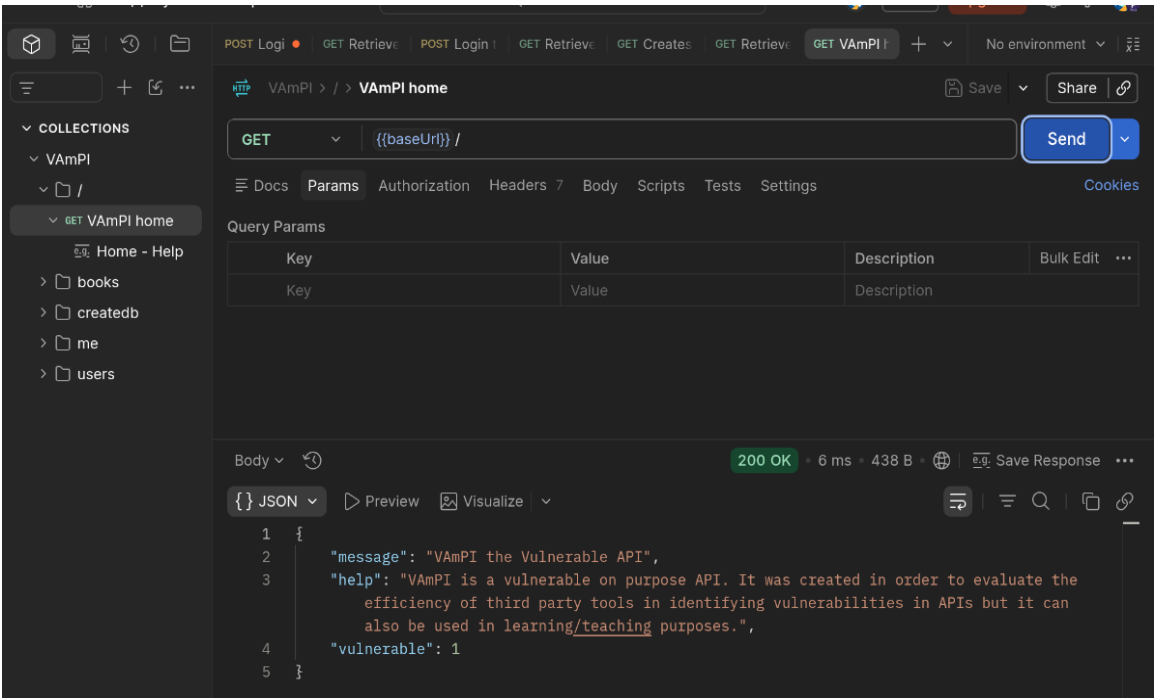


Figure 3 — Unauthenticated Database Reset

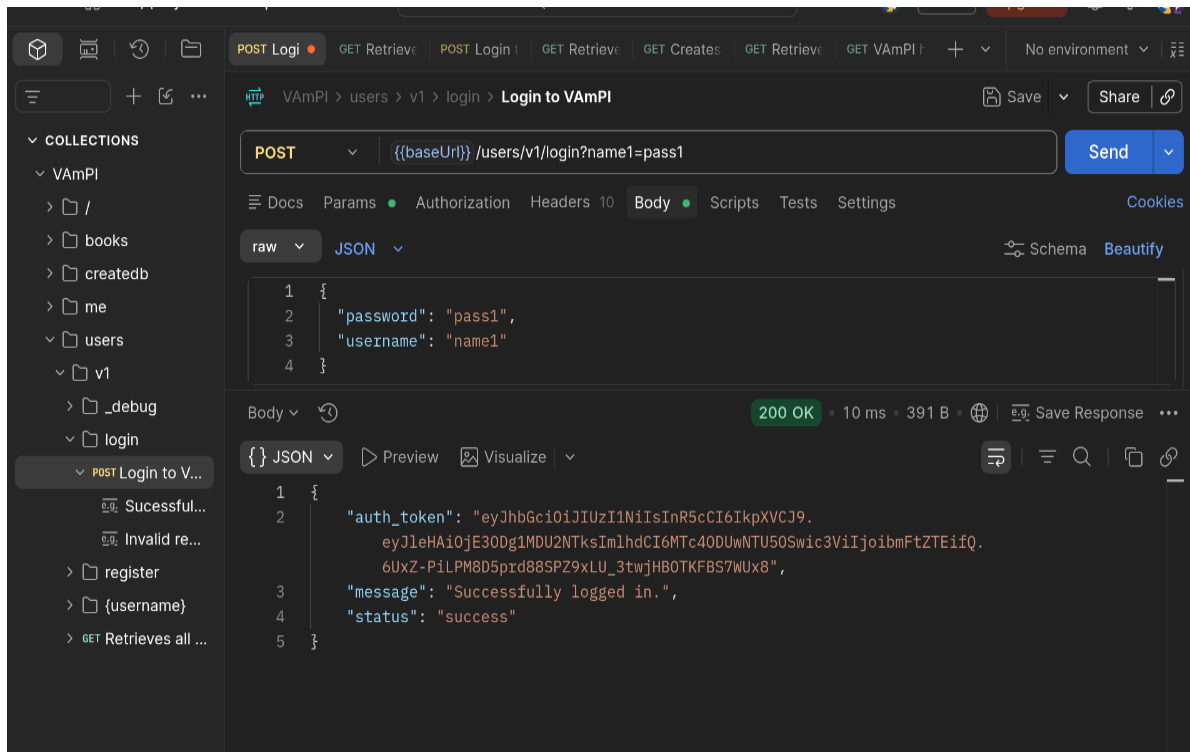


GET /createdb succeeding with no authentication and resetting the database (Finding F1).

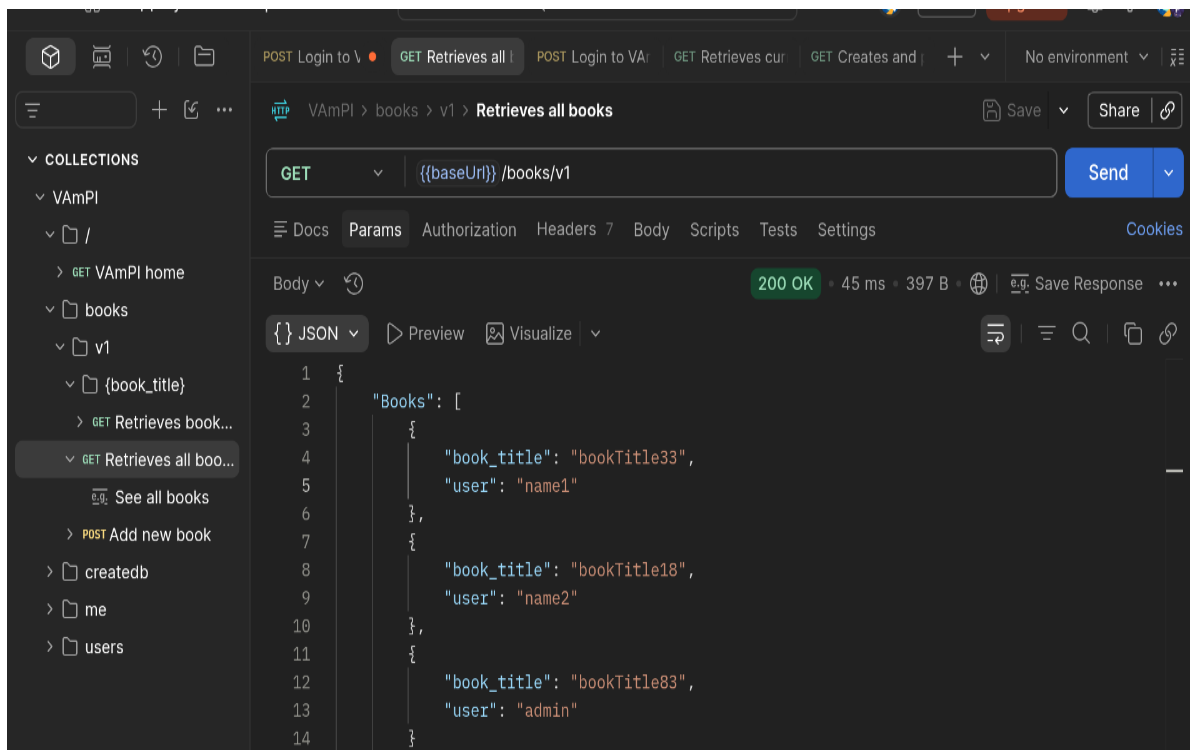
Figure 4 — API Home / Info Endpoint



GET / confirming the target is VAmPI, an intentionally vulnerable API used for this assessment.

Figure 5 — Login and JWT Token Issuance

POST /users/v1/login for user name1, returning a valid JWT auth_token used for subsequent authenticated tests.

Figure 6 — Unauthenticated Book Ownership Listing

GET /books/v1 with no authentication, listing all book titles and their owners, including admin (Finding F5) — this result directly enabled the BOLA test in Figures 7–8.

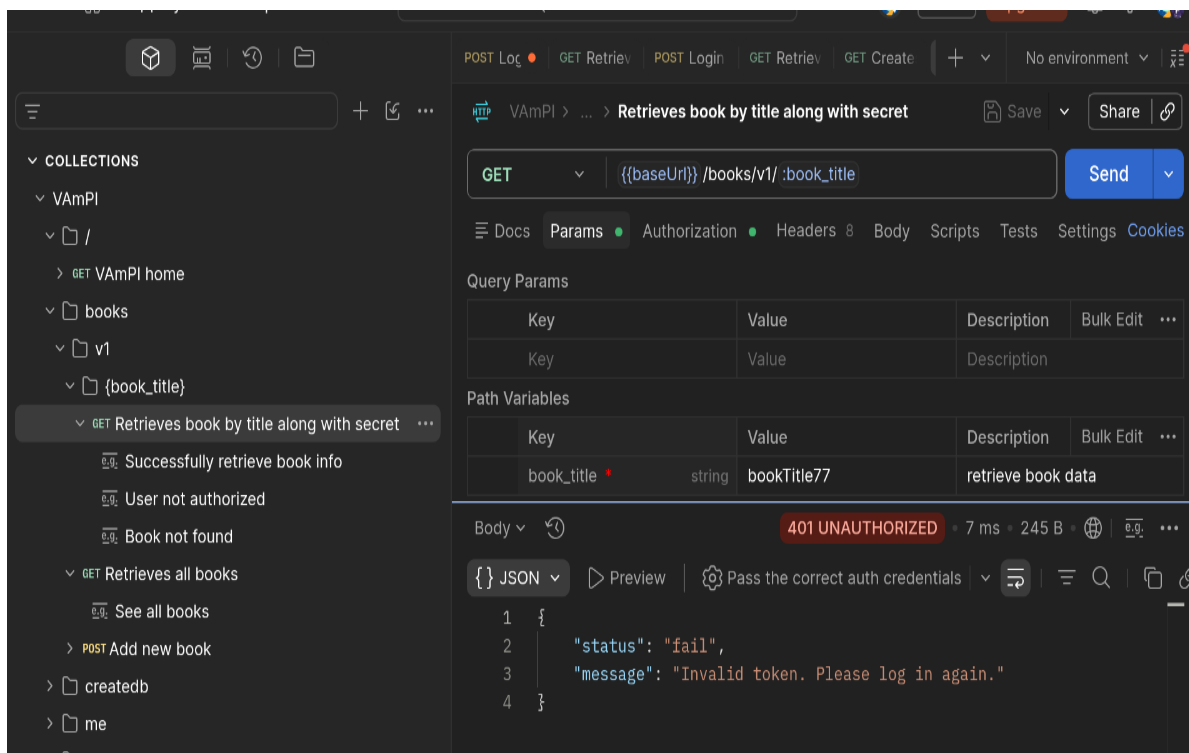
Figures 7–8 — Broken Object Level Authorization (BOLA)

Figure 7 — GET /books/v1/bookTitle83 correctly rejected with 401 Unauthorized when no token is supplied.

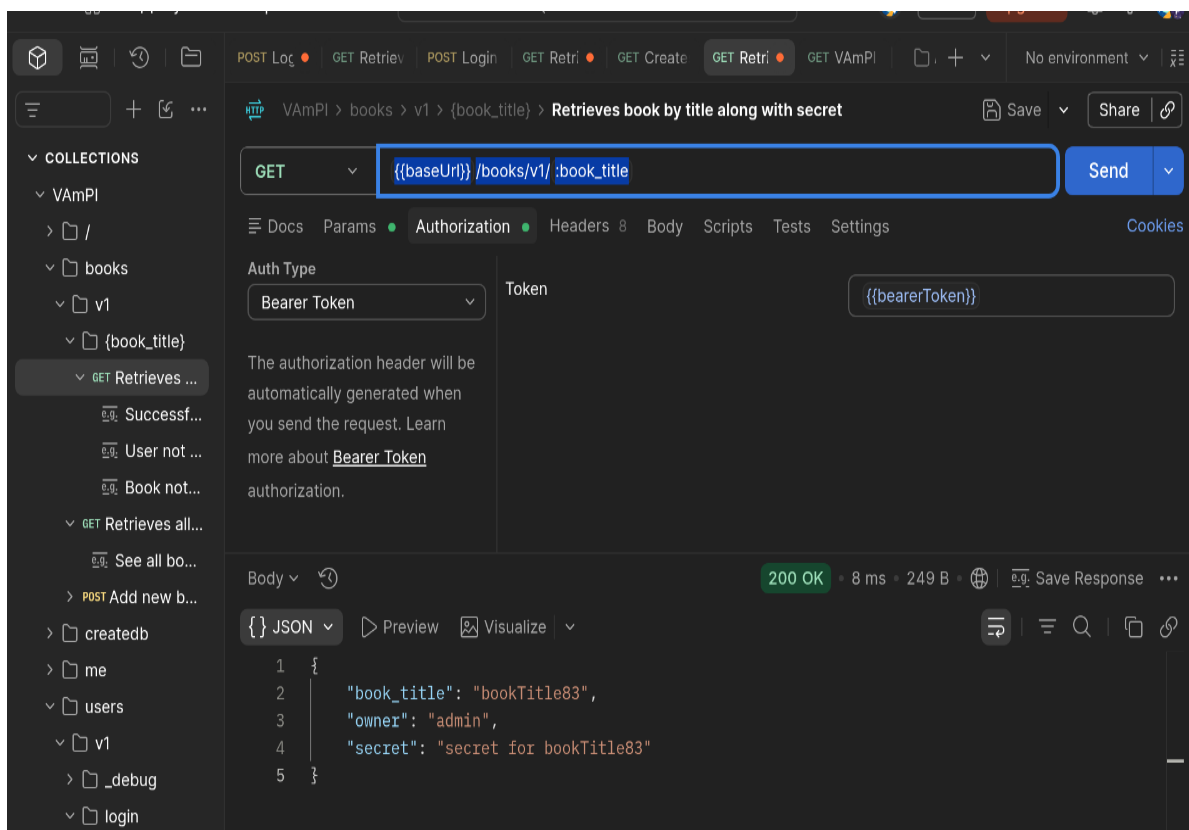
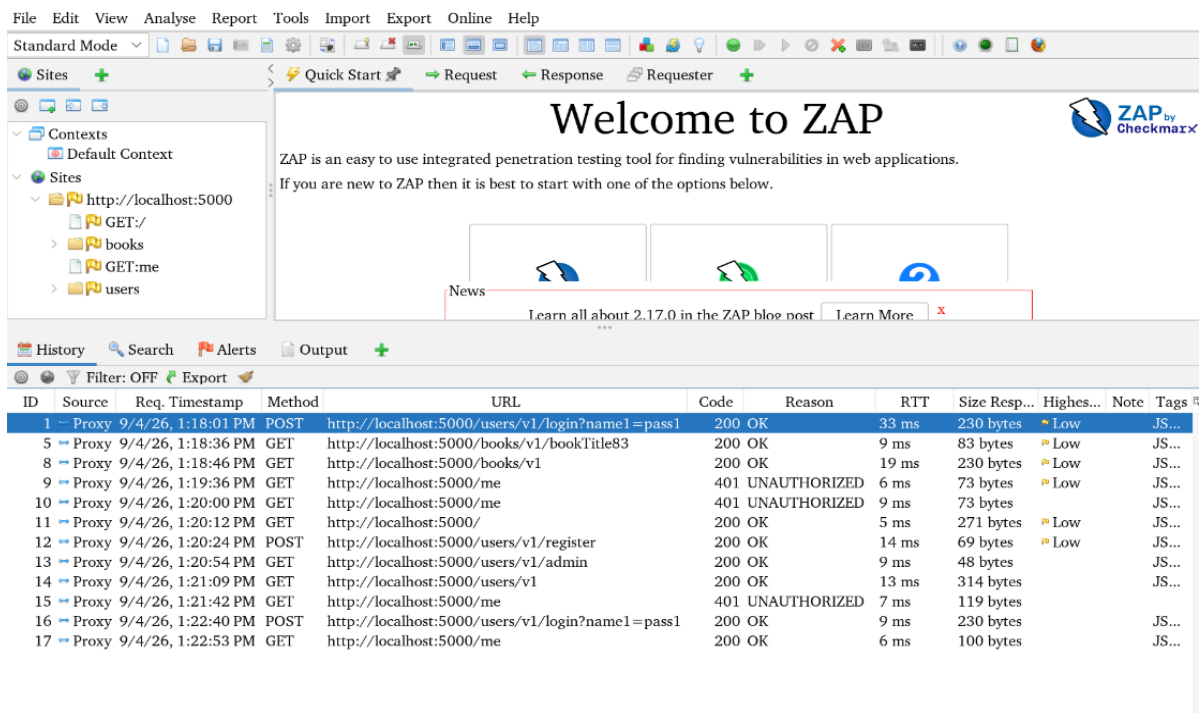


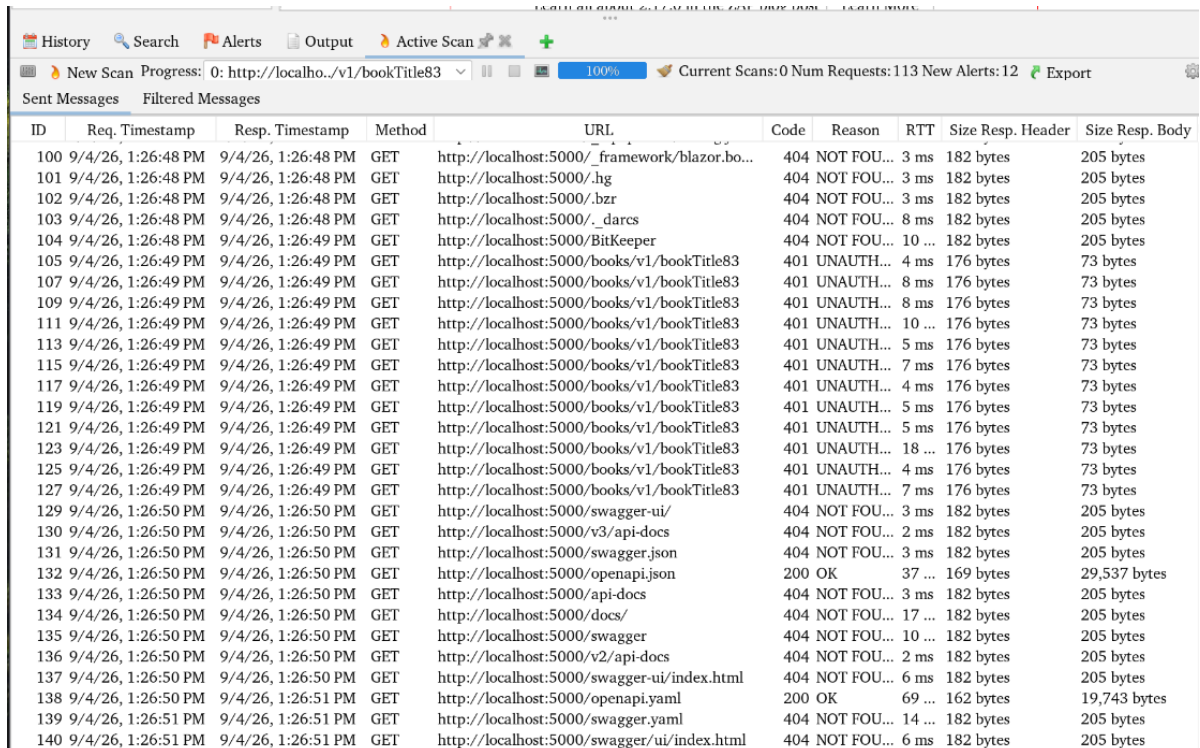
Figure 8 — The same request, now using name1's own Bearer token, successfully returns admin's private book and secret — confirming BOLA (Finding F2).

Figure 9 — Authenticated Traffic Captured in ZAP via Postman Proxy

The screenshot shows the ZAP (Zed Attack Proxy) interface. The top menu bar includes File, Edit, View, Analyse, Report, Tools, Import, Export, Online, and Help. The main window displays a 'Welcome to ZAP' message. On the left, the 'Sites' pane shows a context named 'http://localhost:5000' with sub-sites 'GET:/', 'books', 'GET:/me', and 'users'. The 'History' tab is active, showing a list of requests. The table below represents the data shown in the History tab.

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp...	Highes...	Note	Tags
1	Proxy	9/4/26, 1:18:01 PM	POST	http://localhost:5000/users/v1/login?name1=pass1	200	OK	33 ms	230 bytes	Low		JS...
5	Proxy	9/4/26, 1:18:36 PM	GET	http://localhost:5000/books/v1/bookTitle83	200	OK	9 ms	83 bytes	Low		JS...
8	Proxy	9/4/26, 1:18:46 PM	GET	http://localhost:5000/books/v1	200	OK	19 ms	230 bytes	Low		JS...
9	Proxy	9/4/26, 1:19:36 PM	GET	http://localhost:5000/me	401	UNAUTHORIZED	6 ms	73 bytes	Low		JS...
10	Proxy	9/4/26, 1:20:00 PM	GET	http://localhost:5000/me	401	UNAUTHORIZED	9 ms	73 bytes			JS...
11	Proxy	9/4/26, 1:20:12 PM	GET	http://localhost:5000/	200	OK	5 ms	271 bytes	Low		JS...
12	Proxy	9/4/26, 1:20:24 PM	POST	http://localhost:5000/users/v1/register	200	OK	14 ms	69 bytes	Low		JS...
13	Proxy	9/4/26, 1:20:54 PM	GET	http://localhost:5000/users/v1/admin	200	OK	9 ms	48 bytes			JS...
14	Proxy	9/4/26, 1:21:09 PM	GET	http://localhost:5000/users/v1	200	OK	13 ms	314 bytes			JS...
15	Proxy	9/4/26, 1:21:42 PM	GET	http://localhost:5000/me	401	UNAUTHORIZED	7 ms	119 bytes			
16	Proxy	9/4/26, 1:22:40 PM	POST	http://localhost:5000/users/v1/login?name1=pass1	200	OK	9 ms	230 bytes			JS...
17	Proxy	9/4/26, 1:22:53 PM	GET	http://localhost:5000/me	200	OK	6 ms	100 bytes			JS...

ZAP's History tab showing the full range of requests (including the name1 login and the BOLA request against bookTitle83) after configuring Postman to proxy through ZAP.

Figures 10 and 12 — OWASP ZAP Active Scan Execution

The screenshot shows the ZAP interface with the 'Active Scan' tab active. The top status bar indicates 'New Scan Progress: 0: http://localhost:5000/v1/bookTitle83' and 'Current Scans: 0 Num Requests: 113 New Alerts: 12'. The 'Sent Messages' pane shows a list of requests. The table below represents the data shown in the Sent Messages pane.

ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body
100	9/4/26, 1:26:48 PM	9/4/26, 1:26:48 PM	GET	http://localhost:5000/_framework/blazor.bo...	404	NOT FOU...	3 ms	182 bytes	205 bytes
101	9/4/26, 1:26:48 PM	9/4/26, 1:26:48 PM	GET	http://localhost:5000/_hg	404	NOT FOU...	3 ms	182 bytes	205 bytes
102	9/4/26, 1:26:48 PM	9/4/26, 1:26:48 PM	GET	http://localhost:5000/_bzz	404	NOT FOU...	3 ms	182 bytes	205 bytes
103	9/4/26, 1:26:48 PM	9/4/26, 1:26:48 PM	GET	http://localhost:5000/_darc	404	NOT FOU...	8 ms	182 bytes	205 bytes
104	9/4/26, 1:26:48 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/BitKeeper	404	NOT FOU...	10 ...	182 bytes	205 bytes
105	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	4 ms	176 bytes	73 bytes
107	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	8 ms	176 bytes	73 bytes
109	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	8 ms	176 bytes	73 bytes
111	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	10 ...	176 bytes	73 bytes
113	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	5 ms	176 bytes	73 bytes
115	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	7 ms	176 bytes	73 bytes
117	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	4 ms	176 bytes	73 bytes
119	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	5 ms	176 bytes	73 bytes
121	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	5 ms	176 bytes	73 bytes
123	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	18 ...	176 bytes	73 bytes
125	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	4 ms	176 bytes	73 bytes
127	9/4/26, 1:26:49 PM	9/4/26, 1:26:49 PM	GET	http://localhost:5000/books/v1/bookTitle83	401	UNAUTH...	7 ms	176 bytes	73 bytes
129	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/swagger-ui/	404	NOT FOU...	3 ms	182 bytes	205 bytes
130	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/v3/api-docs	404	NOT FOU...	2 ms	182 bytes	205 bytes
131	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/swagger.json	404	NOT FOU...	3 ms	182 bytes	205 bytes
132	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/openapi.json	200	OK	37 ...	169 bytes	29,537 bytes
133	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/api-docs	404	NOT FOU...	3 ms	182 bytes	205 bytes
134	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/docs/	404	NOT FOU...	17 ...	182 bytes	205 bytes
135	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/swagger	404	NOT FOU...	10 ...	182 bytes	205 bytes
136	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/v2/api-docs	404	NOT FOU...	2 ms	182 bytes	205 bytes
137	9/4/26, 1:26:50 PM	9/4/26, 1:26:50 PM	GET	http://localhost:5000/swagger-ui/index.html	404	NOT FOU...	6 ms	182 bytes	205 bytes
138	9/4/26, 1:26:50 PM	9/4/26, 1:26:51 PM	GET	http://localhost:5000/openapi.yaml	200	OK	69 ...	162 bytes	19,743 bytes
139	9/4/26, 1:26:51 PM	9/4/26, 1:26:51 PM	GET	http://localhost:5000/swagger.yaml	404	NOT FOU...	14 ...	182 bytes	205 bytes
140	9/4/26, 1:26:51 PM	9/4/26, 1:26:51 PM	GET	http://localhost:5000/swagger/ui/index.html	404	NOT FOU...	6 ms	182 bytes	205 bytes

Figure 10 — ZAP Active Scan running against /books/v1/bookTitle83, generating 12 new alerts from 113 requests.

ID	Req. Timestamp	Resp. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Header	Size Resp. Body
231	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/_framework/blazor.bo...	404	NOT FOU...	6 ms	182 bytes	205 bytes
232	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/_hg	404	NOT FOU...	4 ms	182 bytes	205 bytes
233	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/_bzz	404	NOT FOU...	22 ...	182 bytes	205 bytes
234	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/_dars	404	NOT FOU...	9 ms	182 bytes	205 bytes
235	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/BitKeeper	404	NOT FOU...	8 ms	182 bytes	205 bytes
236	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	7 ms	176 bytes	73 bytes
238	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	8 ms	176 bytes	73 bytes
240	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	7 ms	176 bytes	73 bytes
242	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	5 ms	176 bytes	73 bytes
244	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	5 ms	176 bytes	73 bytes
246	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	6 ms	176 bytes	73 bytes
248	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	9 ms	176 bytes	73 bytes
250	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	7 ms	176 bytes	73 bytes
252	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	7 ms	176 bytes	73 bytes
254	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	5 ms	176 bytes	73 bytes
256	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	4 ms	176 bytes	73 bytes
258	9/4/26, 1:31:28 PM	9/4/26, 1:31:28 PM	GET	http://localhost:5000/me	401	UNAUTH...	11 ...	176 bytes	73 bytes
260	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/swagger-ui/	404	NOT FOU...	3 ms	182 bytes	205 bytes
261	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/v3/api-docs	404	NOT FOU...	6 ms	182 bytes	205 bytes
262	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/swagger.json	404	NOT FOU...	3 ms	182 bytes	205 bytes
263	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/openapi.json	200	OK	70 ...	169 bytes	29,537 bytes
264	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/api-docs	404	NOT FOU...	5 ms	182 bytes	205 bytes
265	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/docs/	404	NOT FOU...	6 ms	182 bytes	205 bytes
266	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/swagger	404	NOT FOU...	6 ms	182 bytes	205 bytes
267	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/v2/api-docs	404	NOT FOU...	7 ms	182 bytes	205 bytes
268	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/swagger-ui/index.html	404	NOT FOU...	4 ms	182 bytes	205 bytes
269	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/openapi.yaml	200	OK	62 ...	162 bytes	19,743 bytes
270	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/swagger.yaml	404	NOT FOU...	9 ms	182 bytes	205 bytes
271	9/4/26, 1:31:29 PM	9/4/26, 1:31:29 PM	GET	http://localhost:5000/swagger/ui/index.html	404	NOT FOU...	5 ms	182 bytes	205 bytes

Figure 12 — ZAP Active Scan running against the /me endpoint (current authenticated user), as part of the same authenticated scan pass.

Figure 11 — OWASP ZAP Alerts

Alerts (5)	Full details of any select
- Server Leaks Version Information via "Server" HTTP Response Header Field (Systemic) - GET: http://localhost:5000/ - GET: http://localhost:5000/books/v1 - GET: http://localhost:5000/books/v1/bookTitle83 - GET: http://localhost:5000/me - POST: http://localhost:5000/users/v1/login (name1)({username,password}) - X-Content-Type-Options Header Missing (Systemic) - GET: http://localhost:5000/ - GET: http://localhost:5000/books/v1 - GET: http://localhost:5000/books/v1/bookTitle83 - POST: http://localhost:5000/users/v1/login (name1)({username,password}) - POST: http://localhost:5000/users/v1/register ()({username,password,email,admin}) - Authentication Request Identified - POST: http://localhost:5000/users/v1/login (name1)({username,password}) - Session Management Response Identified - POST: http://localhost:5000/users/v1/login (name1)({username,password}) - User Agent Fuzzer (Systemic) - GET: http://localhost:5000/books/v1/bookTitle83 - GET: http://localhost:5000/books/v1/bookTitle83 - GET: http://localhost:5000/books/v1/bookTitle83 - GET: http://localhost:5000/books/v1/bookTitle83 - GET: http://localhost:5000/books/v1/bookTitle83	You can manually add a y and selecting 'Add ale You can also edit existin

Figure 11 — ZAP Alerts tab listing the automated findings: Server Leaks Version Information, X-Content-Type-Options Header Missing, Authentication Request Identified, Session Management Response Identified, and User Agent Fuzzer (Finding F6).

Figures 13–14 — Swagger UI Documentation

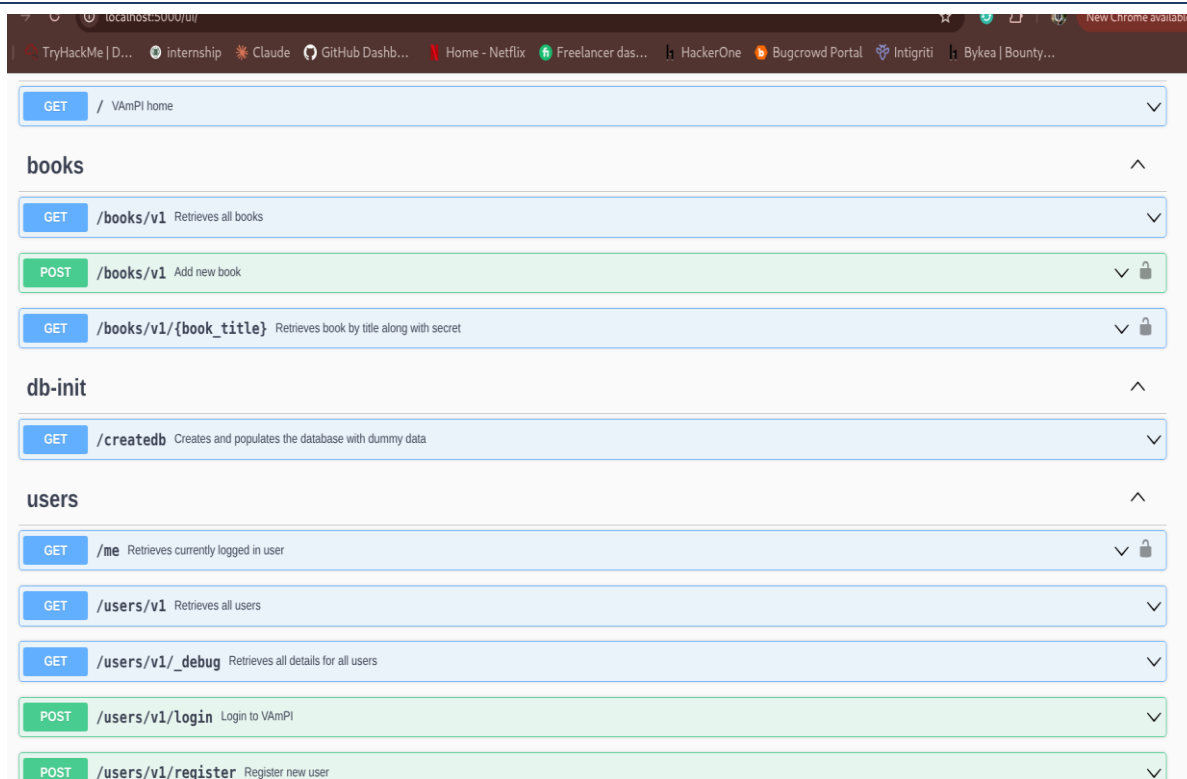


Figure 13 — Swagger UI listing of the books, db-init, and users endpoint groups, used to cross-verify the API map in Section 1.4.

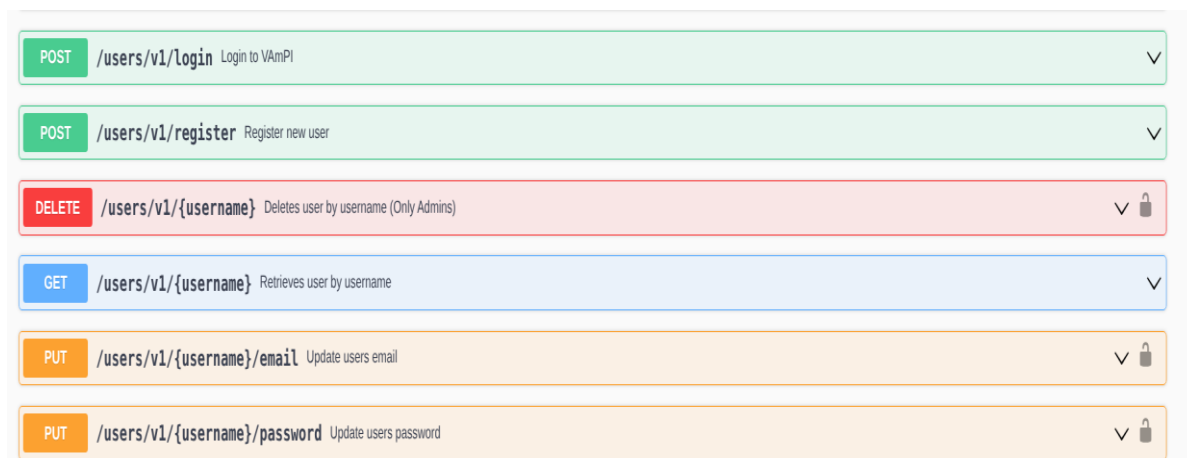


Figure 14 — Swagger UI documentation showing DELETE /users/v1/{username} explicitly restricted to “Only Admins”, the basis for the impact analysis in Finding F8.

5. Recommendations

Access Control (Critical Priority — Findings F1, F2)

- Require authentication on every endpoint that performs a destructive or administrative action, including /createdb; such functions should also be restricted to admin-only roles, not just any authenticated user.
- Implement Object Level Authorization checks on every endpoint that accepts a resource identifier (such as book_title): verify that the requesting user is the owner of, or is explicitly permitted to access, the specific resource requested — not just that they hold a valid token.

Mass Assignment (Critical Priority — Finding F3)

- Use an explicit allow-list of fields accepted during user registration and profile updates; never bind the raw request body directly to a database model.
- Ensure privilege-related fields (such as admin) can only ever be set through a separate, properly authorized administrative process, never through self-service registration.

Data Exposure (Critical Priority — Finding F4; Medium Priority — Finding F5)

- Never return password fields (plaintext or hashed) in any API response, under any circumstance.
- Store passwords using a strong, salted hashing algorithm (e.g. bcrypt or Argon2) rather than plain text.
- Review every “list” style endpoint (such as /books/v1 and /users/v1) and apply the same authentication and object-level authorization controls used on their corresponding “detail” endpoints, to remove the inconsistency found in this assessment.

Security Headers (Low Priority — Finding F6)

- Suppress or generalise the Server response header to avoid disclosing specific software versions.
- Add the X-Content-Type-Options: nosniff header to all API responses.

Session Management (Informational — Finding F7)

- Review the JWT token expiry policy; balance security against usability by choosing an expiry appropriate to the sensitivity of the API and the expected client behaviour (e.g. 15–30 minutes with refresh tokens, rather than 60 seconds).

General

- Conduct a full audit of every admin-only endpoint (such as DELETE /users/v1/{username}) to confirm the privilege check cannot be bypassed by a self-escalated account created via Mass Assignment (Finding F8).
- Re-test all findings after remediation using the same methodology used in this assessment: OpenAPI-driven mapping in Postman, manual authorization testing across multiple accounts, and an authenticated OWASP ZAP Active Scan.

6. Conclusion

Week 7 API security testing of VAmPI mapped the complete API surface from its OpenAPI specification and combined manual authorization testing in Postman with an automated OWASP ZAP scan. This produced 8 findings: 4 Critical, 1 Medium, 1 Low, and 2 Informational. The Critical findings — an unauthenticated database reset, a Broken Object Level Authorization flaw, a Mass Assignment privilege-escalation path, and an Excessive Data Exposure issue returning every user's plaintext password — together point to a systemic pattern: the API frequently authenticates who a user is but does not consistently authorize what that user should be allowed to do or see.

This pattern was reinforced by the automated ZAP scan, which, once configured to see authenticated traffic via a Postman proxy, confirmed the same endpoints while additionally surfacing lower-severity configuration issues (missing security headers). The combination of manual, authorization-focused testing and automated scanning proved complementary: the automated scan alone (before authentication was configured) returned zero alerts, while manual testing uncovered the most severe issues — illustrating why automated API scanning must always be paired with authenticated, logic-aware manual testing rather than used on its own.